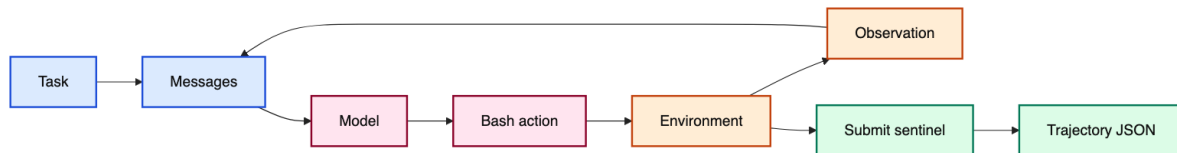


第 08 讲: mini-SWE-agent: 极简 Agent 的上限

Coding Agent Notes (June 2026)

1 核心图景

mini-SWE-agent 的价值不在于“功能少”，而在于它给 coding agent 设定了一个很清楚的下限：如果一个系统只保留 model、bash action、execution environment、linear messages 和 submission condition，它还能不能完成真实软件工程任务？



这个图可以读成一句话：任务被写进 messages，模型输出 bash action，环境执行命令并返回 observation，observation 继续追加到 messages；直到环境看到提交哨兵，系统把轨迹保存为 JSON。

mini-SWE-agent 的核心主张：

一个可用的 coding agent 不一定先从复杂工具平台开始。

先把最小 loop 做清楚，才能判断能力来自模型、shell、环境，还是来自 scaffold。

这和前面几讲的关系很直接。#05 讲过 harness 的通用词汇，#06 讲过 SWE-agent 如何设计专用 ACI，#07 讲过 Agentless 如何把 repo repair 做成 pipeline。mini-SWE-agent 则问一个更朴素的问题：如果不做复杂 ACI、不做多阶段定位、不做平台事件流，只给模型一个 shell，它的上限在哪里？

1.1 一个最小修复轨迹

设想一个 SWE-bench issue：某个库的配置解析函数在空字符串输入时返回错误类型。mini-SWE-agent 不会先构造 repo map，也不会调用专门的 search tool。它会把 issue 描述写进 user message，然后让模型自己选择 bash 命令。

一个典型轨迹会长成这样。第一轮，模型用 ls 和 rg 找项目结构和关键词。第二轮，它用 sed -n 或 nl -ba 读取可疑文件的局部代码。第三轮，它写一个很小的 reproduction script 或直接运行相关测试。第四轮，它用 shell 命令修改源文件，可以是短脚本、patch、formatter 或项目已有命令。第五轮，它重新跑测试，确认 return code 和 output。最后，它生成 patch 并用提交哨兵结束。

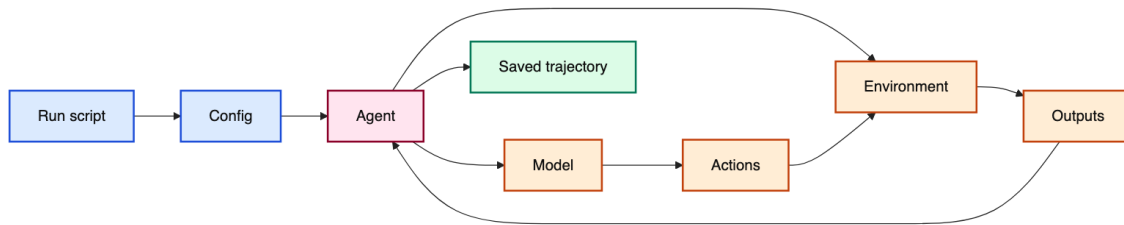
这条轨迹里没有任何神秘 agent 技巧。关键能力都暴露在模型是否能正确使用 shell：会不会找文件、会不会读局部上下文、会不会写最小测试、会不会谨慎编辑、会不会解释失败输出、会不会在提交前检查 diff。

mini 的教学价值：

它把 coding-agent 能力还原成一组可观察的 shell 操作能力，因此很适合判断模型本身是否真的会在 repo 里工作。

2 极简不是没有结构

mini-SWE-agent 的“minimal”不是把工程结构删光，而是把结构压缩到四个组件：Agent、Model、Environment、Run/config。每个组件的边界很窄，因此读代码时很容易看出责任归属。



组件	主要职责	对系统的意义
Agent	控制 loop、维护 messages、保存轨迹	决定状态如何流动
Model	查询 LM、解析 action、格式化 observation	决定接口语言
Environment	执行 command、返回 stdout/rc	决定世界如何反馈
Run/config	绑定组件、设置 prompt/limit/backend	决定使用场景

这个结构非常适合教学。复杂 agent 平台往往把工具注册、权限、事件、runtime、前端、评测、日志混在一起看；mini-SWE-agent 把它们拆开成几个小对象。你可以替换 model，不改 agent loop；替换 Docker environment，不改 model；换一个 run script，也不必重写核心 agent。

Insight. mini-SWE-agent 不是“没有架构”，而是架构的自由度被压到最低。它让我们能观察：一个 coding agent 到底需要哪些不可再约的部分。

3 DefaultAgent loop

核心代码在 `src/minisweagent/agents/default.py`。从控制流看，`DefaultAgent.run(task)` 先渲染 `system message` 和 `user task message`，然后进入循环。每一轮调用 `step()`；而 `step()` 的主体可以压缩成：

```

step() = query() + execute_actions()
query() 调模型并追加 assistant message;
execute_actions() 执行动作并追加 observation messages。

```

3.1 run: 初始化和停止

`run()` 的第一件事是把任务写入模板。模板来自 `config`，可以包含任务描述、系统信息、命令规则、提交规则等。接着 `loop` 不断调用 `step()`。停止条件也很简单：如果最后一条 `message` 的 `role` 是 `exit`，循环结束，并返回最后一条 `message` 的 `extra` 字段。

这个停止条件很小，但很重要。它把“任务是否完成”变成轨迹里的一个显式消息，而不是隐藏在某个外部变量里。后续保存 `.traj.json` 时，完成状态、`submission`、`cost`、`message history` 都能放在同一个对象里。

3.2 query: 预算先于模型

query() 在调用模型之前检查 step limit、cost limit 和 wall-time limit。默认 agent config 有 step_limit、cost_limit、wall_time_limit_seconds。如果超限，它会抛出 LimitsExceeded 或 TimeExceeded，并生成一条 exit message。

这说明预算控制不是附属功能，而是 agent loop 的一部分。一个长期编码 agent 如果不把预算作为一等状态，就很难在 batch benchmark 或产品环境中稳定运行。

3.3 InterruptAgentFlow

mini-SWE-agent 用异常承载结构化中断：Submitted、LimitsExceeded、TimeExceeded、FormatError、UserInterruption 都继承自 InterruptAgentFlow。这些异常不是普通 crash，而是带着 messages 回到主循环。

这是一种很小但很干净的设计。模型格式错了，可以把格式错误作为 observation 追加给模型；用户中断了，可以把用户意见写入 trajectory；提交成功了，可以通过 Submitted 写入 exit message。控制流和可追踪历史因此合在一起。

4 Bash 是唯一动作空间

mini-SWE-agent 最鲜明的选择是：默认把 action space 收缩成 bash。模型不需要选择 search、open、edit、submit 等多个专用工具，而是输出一个或多个 bash commands。

动作设计	优点	代价
专用工具集	操作语义清楚，反馈可定制	scaffold 复杂，模型适配强
Bash-only	通用、可移植、模型负担集中	命令更脆，安全边界更重要
Pipeline stages	可审计、可批量验证	交互探索较弱

这里的 bash-only 不是说没有接口约束。v2 默认模型类 LitellmModel 仍然注册了一个名为 bash 的工具，工具参数是 command。如果模型不支持或不适合 tool call，也可以用 LitellmTextbasedModel 从 mswea_bash_command 代码块中用 regex 解析 action。

4.1 FormatError 也是反馈

如果模型没有给 tool call，或者 tool 名不叫 bash，或者参数里没有 command，parser 会抛出 FormatError。text-based 模式也类似：如果没有正好一个 action block，就会生成格式错误反馈。

这点很关键。极简系统不能靠复杂 GUI 或 tool framework 自动纠错，所以格式错误必须进入模型可见的 trajectory。mini-SWE-agent 的做法是把格式错误包装成 message，让下一轮模型知道自己违反了动作协议。

Insight. Bash-only 的真正含义不是“没有工具”，而是“只有一个通用工具”。这把工具选择问题压到最低，同时把环境操作能力的责任交给模型。

4.2 Bash 把工具生态交给操作系统

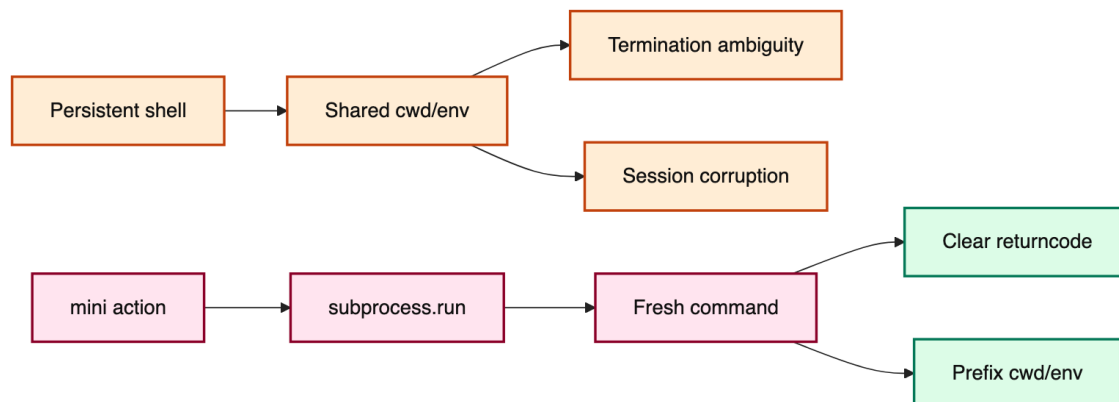
专用 ACI 的思路是为 agent 做一组更友好的命令。mini 的思路相反：既然软件工程师已经用 shell 把搜索、编辑、测试、格式化、构建、打包串起来，那么 agent 也可以先学会使用这些既有工具。

任务	Bash 工具形态	模型需要掌握什么
找上下文	rg, find, git grep	关键词选择和范围控制
读代码	sed -n, nl -ba, head/tail	控制输出长度
改文件	脚本、patch、formatter	保持 diff 小且可审计
验证	pytest, build, custom script	从 return code 和日志定位失败

这种设计让 mini 不必为每种语言、框架和编辑器写专门工具。但它也要求模型懂得“少量、可恢复、可验证”的 shell 工作方式。模型如果只会一次性输出巨大命令，或者不懂如何把输出限制到可读范围，bash-only 反而会放大错误。

5 为什么不保留 shell session

许多 agent 会保留一个常驻 shell session，把模型命令一行行粘进去。mini-SWE-agent 刻意反过来做：每个 action 都独立执行。Local environment 用 subprocess.run；Docker environment 用 docker exec；每次都有明确的 return code 和 output。



5.1 常驻 shell 的隐藏复杂性

常驻 shell 看起来更接近真实终端，因为 cd、export、激活 virtualenv 等状态可以保留。但它带来三个麻烦：第一，系统很难可靠判断命令什么时候结束；第二，坏命令可能杀掉 shell；第三，中断命令可能污染后续输出。

这些问题在单次 demo 中不明显，但在几百个 benchmark instances 上会变成工程噪声。mini-SWE-agent 选择牺牲一部分便利性，换取每一步执行的可边界化。

5.2 代价：状态不持久

独立 action 的代价也很实际：工作目录和环境变量不会自动延续。配置和 prompt 因此明确提醒模型：每次 action 都在新的 subshell 中执行，如果需要目录或环境变量，要把它们写在每条命令前面。例如把 cd /testbed && pytest 当成一个完整动作，而不是先 cd，下一轮再 pytest。

这个代价不是 bug，而是设计取舍。它把隐式 session state 变成显式 command prefix。对评测和安全来说，这通常更容易复现。

6 Observation 和 trajectory

一个 coding agent 的质量，很大程度取决于它给模型看什么反馈。mini-SWE-agent 的 observation 很朴素：return code、output、exception info。如果输出太长，config 可以把它截成 head/tail，并告诉模型中间省略了多少字符。

字段	含义	为什么重要
returncode	命令成功或失败	给模型明确执行信号
output	stdout/stderr 合并文本	承载测试错误和文件信息
exception_info	timeout 或执行异常	区分命令失败和环境失败
raw_output	未格式化原始输出	便于轨迹审计

这些 observation 会被 model 类格式化成下一条 message。于是 messages 同时是工作记忆，也是 trajectory。README 中强调，mini 的 history 是完全线性的，每一步只追加消息；所以“传给模型的上下文”和“保存下来的轨迹”没有本质差别。

6.1 轨迹文件

.traj.json 里保存了完整运行：model cost、API calls、agent/model/environment config、mini version、exit status、submission、messages，以及 trajectory_format。这让 mini-SWE-agent 很适合做调试和微调数据。你不需要还原复杂 event stream，只要读 messages 就能知道模型看到了什么、做了什么、环境回了什么。

这种线性历史也有局限。长任务会不断膨胀上下文；没有默认 history processor，就无法自动摘要、分层记忆或选择性遗忘。mini 的答案是先保持透明，再让使用者在需要时扩展。

6.2 为什么线性轨迹适合做数据

对于训练和评测，线性轨迹有一个很强的优点：每一轮模型看到的上下文，几乎就是保存文件里的前缀。也就是说，如果要从一次成功修复中学习，你不需要重建复杂状态机；只需要读取 messages，就知道 prompt、assistant action、tool result 的顺序。

数据对象	mini 中的表示	用途
模型输入	messages prefix	复现下一步决策
模型输出	assistant message + actions	学习 command planning
环境反馈	observation message	学习从错误恢复
最终结果	exit status + submission	连接轨迹和评价

这也是 mini 适合做 baseline 的原因。它不会把大量行为藏在 UI、后台状态或外部工具对象里。轨迹虽然朴素，但证据链完整。

7 Config 是小型产品面

mini-SWE-agent 的复杂性主要不在核心 loop，而在配置。YAML 顶层通常分为 agent、environment、model、run。这让同一个 loop 可以变成 CLI、本地开发工具、SWE-bench runner 或自定义实验 harness。

配置区	常见内容	改变什么
agent	templates, limits, output path	行为规则和预算
model	model name, class, kwargs	模型接口与成本追踪
environment	cwd, env, timeout, backend	命令执行位置与隔离
run	output, startup command	批处理和场景绑定

Prompt 模板用 Jinja 渲染，可以引用 task、系统信息、环境变量、配置变量、模型调用次数和已花费成本。这样，prompt 不是硬编码字符串，而是和运行环境相连的配置面。

7.1 InteractiveAgent

mini 还提供 InteractiveAgent，把用户放进 loop。它有三种模式：human 表示用户命令立即执行；confirm 表示模型命令需要用户确认；yolo 表示模型命令直接执行。用户还可以用 slash commands 在模式间切换。

这说明 human-in-the-loop 不一定需要重写核心 agent。只要扩展 query() 和 execute_actions()，就能在相同 messages 和 environment 上增加确认、拒绝、追加任务、确认提交等行为。

8 环境后端是可替换边界

极简 agent 最容易被误解成只能在本机执行命令。实际上 mini-SWE-agent 把 execution backend 做成可替换的 environment class：local、Docker、Singularity/Apptainer、SWE-ReX Docker、SWE-ReX Modal、bubblewrap、ConTree 都是同一个抽象的不同实现。

Local。

LocalEnvironment 直接在宿主机上用 subprocess.run 执行命令。它适合本地 CLI 和快速实验，但没有隔离，权限边界要靠用户和操作系统自己管理。

Docker / Singularity。

Docker environment 启动容器，然后用 docker exec 在容器中执行每条命令。Singularity/Apptainer 适合 HPC 场景。对 SWE-bench 来说，容器后端很关键，因为每个 instance 需要对应的依赖和仓库状态。

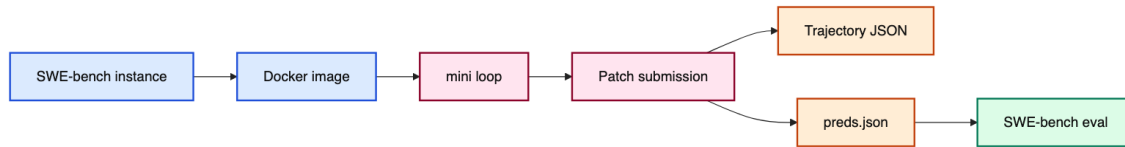
SWE-ReX / Modal / sandbox backends。

SWE-ReX、Modal、bubblewrap、ConTree 等后端说明 mini 的核心 loop 并不关心命令到底在哪里执行。只要 environment 能接收 action、返回 output 和 return code，agent 就能继续工作。

Insight. mini-SWE-agent 的可扩展性来自边界小，而不是功能多。Environment 可以变复杂，但 Agent loop 不必变复杂。

9 SWE-bench 证明它不是玩具

mini-SWE-agent repo 提供 SWE-bench batch 和 single-instance runner。batch 命令可以选择 subset、split、slice、filter、workers、model、config、environment class；single-instance 命令用于调试一个具体任务。



SWE-bench config 把工作目录设成 /testbed，默认 environment class 是 Docker，timeout 是 60 秒，step limit 是 250，cost limit 是 3，默认模型示例是 anthropic/claude-sonnet-4-5-20250929，并设置 temperature: 0.0 和 parallel_tool_calls: true。

9.1 提交协议

SWE-bench 场景中，模型被要求先生成 patch.txt，检查 patch 只包含目标源文件，然后用精确命令提交：

```
echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && cat patch.txt
```

环境看到输出第一行是 COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT 且 return code 为 0，就抛出 Submitted，并把后续文本作为 submission。batch runner 再把每个 instance 的 submission 写入 preds.json，供 SWE-bench harness 或 cloud evaluation 使用。

9.2 每个 instance 都有证据

runner 会为每个 SWE-bench instance 保存 trajectory。即使任务失败，也可以看到 exit status、异常、模型输出、命令输出和最终 submission。这是 mini 作为研究 baseline 的重要性质：它不只给一个分数，还给可复查的过程证据。

10 它到底测到了什么能力

mini-SWE-agent 的结果经常被用来说明：当模型足够强时，复杂 scaffold 未必是性能的主要来源。README 提到 mini 在 SWE-bench Verified 上有超过 74% 的成绩。这类结果不能简单解读成“bash-only 永远最好”，但它确实说明了一个很强的事实：许多 repo repair 能力可以由模型的代码理解、shell 使用、测试解释和局部修改能力共同完成。

观察到的成功	更可能说明	不一定说明
mini 能修 SWE issue	模型能用 shell 完成定位-修改-验证	专用工具完全没价值
轨迹简单可复现	scaffold 噪声低	长任务上下文已解决
bash-only 可扩展	环境抽象足够通用	安全边界天然充分

所以 #08 的结论不是“产品级 agent 都应该只保留 bash”，而是：mini 是一个很强的 lower-bound harness。任何更复杂的设计都应该能解释自己比 mini 多出来的复杂度买到了什么。

11 三种降低复杂度的路线

mini-SWE-agent、SWE-agent 和 Agentless 都在处理同一个核心矛盾：coding agent 需要足够强的环境操作能力，但 scaffold 太复杂又会让系统难以调试。三者的答案不同。

系统	降低哪种复杂度	保留什么能力
mini-SWE-agent	工具和状态复杂度	模型用 shell 自主探索
SWE-agent	人机接口歧义	专用命令和紧凑反馈
Agentless	在线决策复杂度	批量定位、修复、验证

mini 的路线是把工具面压到一个 bash action。它不替模型设计 search、edit、submit 等细工具，而是让模型在 shell 里组合系统已有能力。因此它最适合回答：模型本身能不能像工程师一样使用终端？

SWE-agent 的路线是保留在线 agent loop，但优化 action interface。它承认模型需要和电脑反复交互，因此把命令设计成更简单、更可解析、更有信息密度的 ACI。它最适合回答：如果给模型更好的工具语言，agent trajectory 会不会更稳？

Agentless 的路线更极端：它不让模型自由控制在线 loop，而是把 repo repair 固定成 localization、repair、validation。它最适合回答：如果任务可以被拆成候选生成和候选选择，我们能不能用 batch pipeline 减少轨迹漂移？

Insight. 这三种系统不是互相否定，而是在不同维度上削减复杂度。mini 削工具，SWE-agent 削接口歧义，Agentless 削在线控制自由度。

12 什么时候需要超过 mini

mini 是很强的下限，但不是所有场景的终点。一个有用的判断方式是：如果失败来自模型不会使用 shell，那么换更复杂 scaffold 未必解决问题；如果失败来自 shell 表达不适合任务，那么就该给系统加工具。

失败来源	mini 暴露出的症状	可能需要的扩展
上下文太大	反复 grep、读错文件、超长输出	repo map / retrieval
编辑太脆	sed/perl 命令误改或 diff 太大	structured edit tool
需要 UI 证据	shell 测试不能覆盖视觉状态	browser / Playwright tool
安全风险高	命令权限过大、误删风险	sandbox / permission gate
协作需求强	用户要审阅、确认、改方向	human-in-loop workflow

这张表也解释了为什么 mini 的成功并不会否定 SWE-agent、OpenHands、Aider 或产品级 Codex。mini 证明了最小 loop 的上限很高；更复杂系统应该围绕它无法优雅解决的问题增加结构，而不是重新发明一个更难调试的 loop。

Insight. 工程上最稳的路线通常不是在最开始就做大平台，而是先跑通 mini-style baseline，再按失败证据加 context、tool、permission、UI 或 collaboration 层。

13 局限与开放问题

Bash 很强，但不总是好用。

bash 可以搜索文件、运行测试、编辑文件、调用语言工具、安装依赖，因此覆盖面很广。但 bash 命令也容易变长、变脆、难以审计。专用 edit 工具、browser 工具、structured search 工具在很多产品场景里仍然有价值。

无持久 session 减少噪声，也增加负担。

每条命令独立执行让 return code 清楚、sandbox 替换容易，但模型必须记得每次 prefix cd 或环境变量。对简单 repo repair 这通常可接受；对需要长时间交互式状态的任务，可能会变麻烦。

线性历史透明，但会增长。

线性 messages 非常适合调试和微调，因为 trajectory 就是 prompt history。但长任务会遇到上下文预算。mini 默认不提供复杂 history processor，这既是优点，也是扩展点。

安全不是靠 minimal 自动获得。

Local environment 没有隔离；Docker、bubblewrap、ConTree 等可以提高隔离，但它们仍需要正确配置。bash-only 把大量能力暴露给模型，因此 permission、sandbox、confirmation mode 仍然是产品级系统必须处理的问题。

14 系统设计启发

mini-SWE-agent 给我们一个很实用的设计准则：先从最小 loop 开始，然后只在复杂度能买到明确收益时加复杂度。

1. 如果任务可以通过 shell 搜索、编辑、运行测试完成，先用 bash-only baseline 判断模型本身能力。
2. 如果失败主要来自命令不稳定或安全风险，优先加强 environment 和 permission，而不是先加多 agent。
3. 如果失败来自上下文管理，考虑 history processor、repo map 或 retrieval，而不是把所有责任都交给 prompt。
4. 如果失败来自操作不够可控，再引入专用 ACI 工具，例如结构化 edit、browser、PR、CI 或 notebook 工具。

这也是 mini 和 SWE-agent、Agentless 的互补关系。SWE-agent 告诉我们专用 interface 可以提升操作质量；Agentless 告诉我们有些 repair 可以变成 batch search；mini 告诉我们在加入这些复杂性之前，先确认最小 shell loop 的上限。

15 最终 takeaway

mini-SWE-agent 可以压缩成三句话。

- 它把 coding agent 的核心压缩成 model、bash、environment、messages 和 submission sentinel。
- 它用无常驻 shell、线性轨迹和可替换 environment，让极简系统也能批量跑真实 SWE-bench 任务。
- 它是判断模型能力和 scaffold 价值的下限基准：如果复杂系统比它强，必须说明多出来的复杂度买到了什么。

因此，读 mini-SWE-agent 最重要的不是记住某个 leaderboard 数字，而是理解一个工程原则：**先证明最小系统不够，再引入复杂系统**。对 coding agent 来说，复杂度应该服务于信息、验证、安全和

人机协作，而不是为了显得更像一个平台。

使用 mini-style baseline 时的检查表：

如果最小 loop 已经能稳定完成任务，先不要加平台层；
如果失败来自上下文，补 repo map 或 history processor；
如果失败来自动作脆弱，补 structured edit 或专用 tool；
如果失败来自风险和协作，补 sandbox、permission 和 human review。

参考资料

- [1] [mini-SWE-agent repository](#). default agent loop, environment backends, model interfaces, configs, and runners
- [2] [mini-SWE-agent documentation](#). control flow, config, environments, output format
- [3] [SWE-bench](#). benchmark context and evaluation target used by mini-SWE-agent